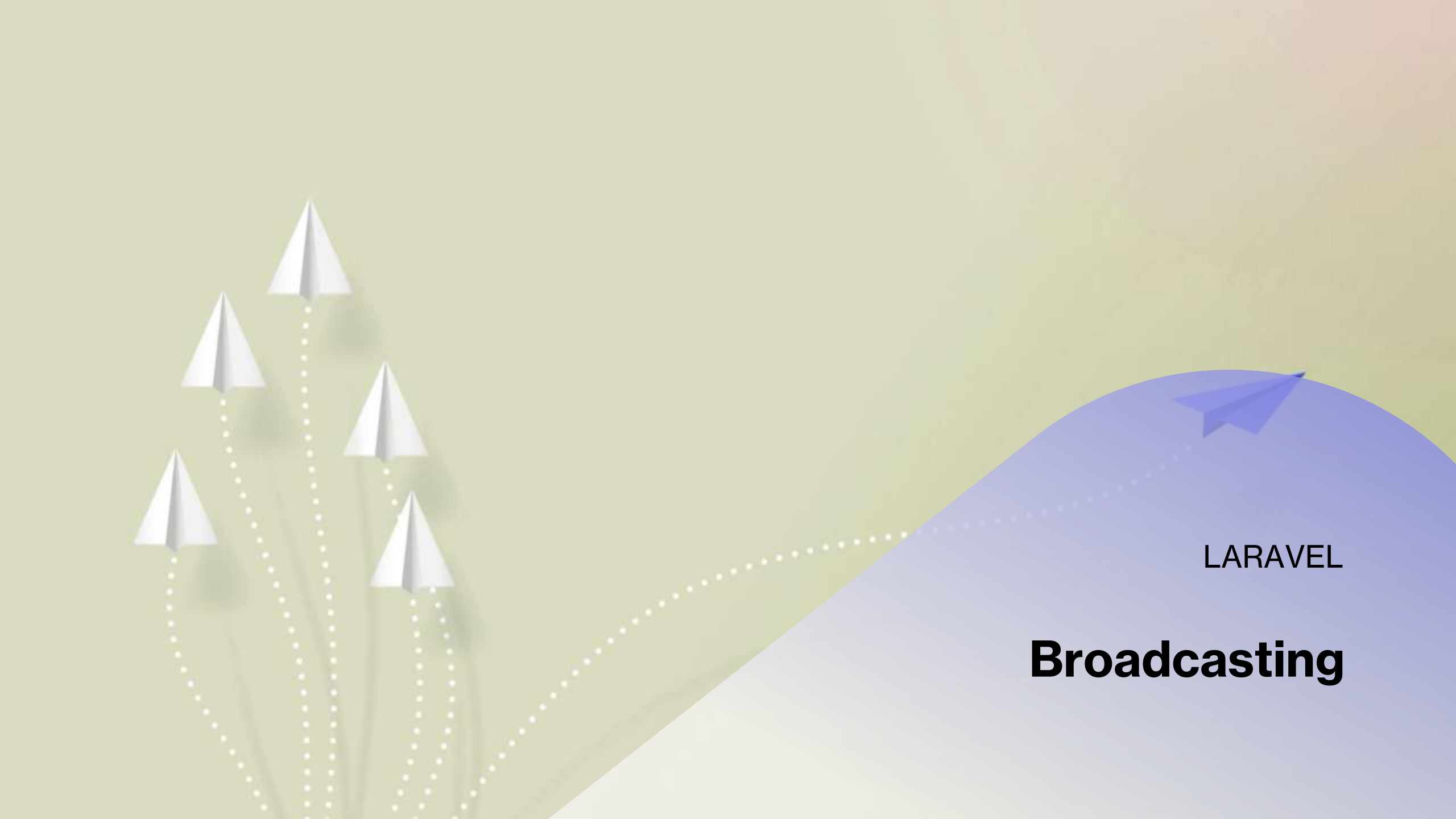




LARAVEL

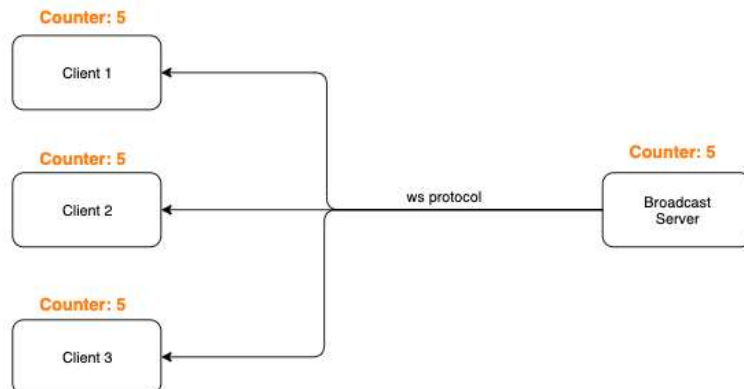
Broadcasting

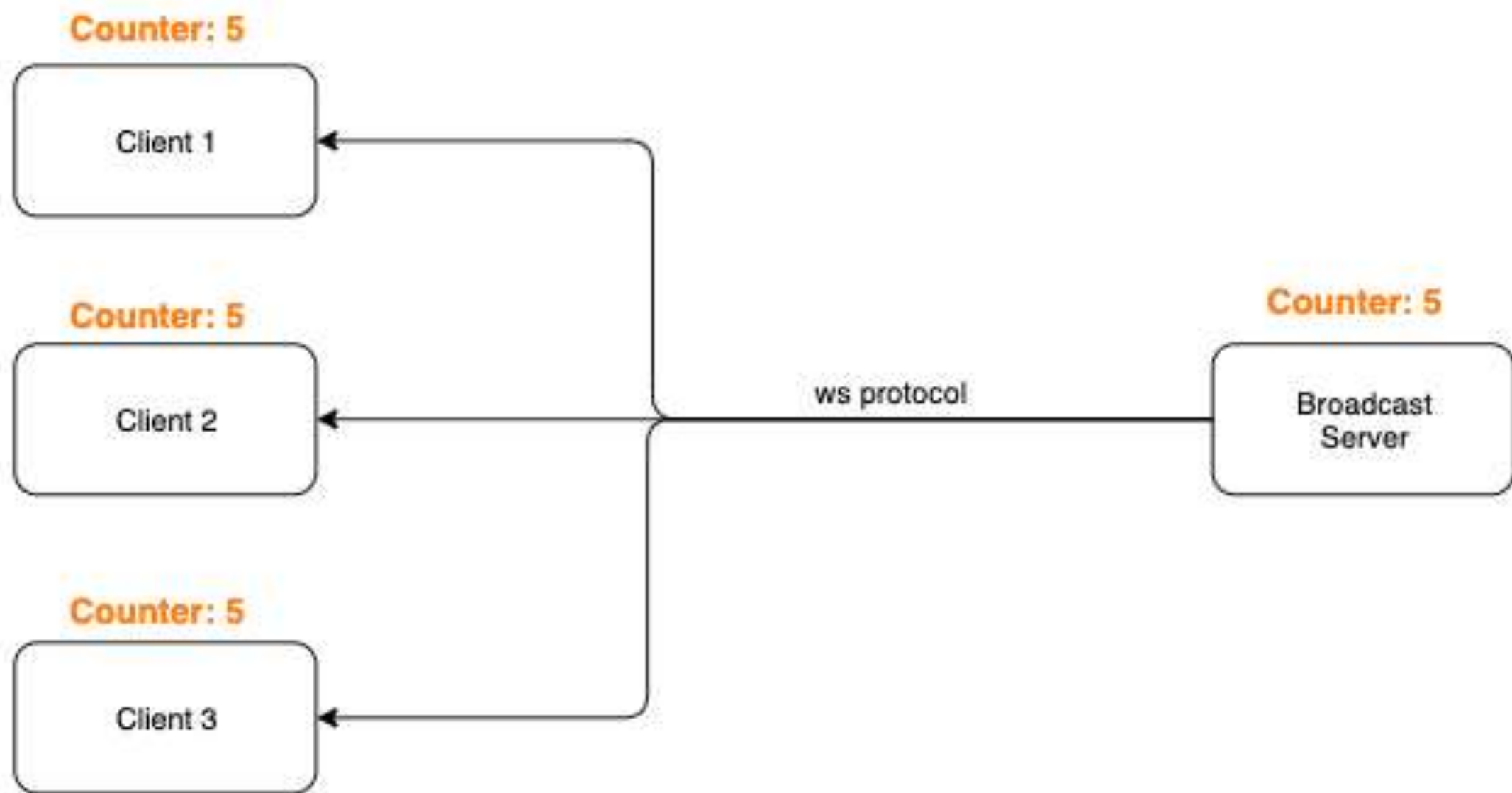
Introduction

Dans de nombreuses applications web modernes, les **WebSockets** sont utilisées pour mettre en œuvre des interfaces utilisateur en temps réel et actualisées en direct. Lorsque des données sont mises à jour sur le serveur, un message est généralement envoyé via une connexion **WebSocket** pour être traité par le client. Les WebSockets offrent une alternative plus efficace à l'interrogation continue du serveur de votre application pour les changements de données qui doivent être reflétés dans votre interface utilisateur.

Introduction

Le concept de base de la diffusion est simple : les clients se connectent à des canaux nommés sur le front-end, tandis que votre application Laravel diffuse des événements vers ces canaux sur le back-end. Ces événements peuvent contenir toutes les données supplémentaires que vous souhaitez mettre à la disposition du frontend.

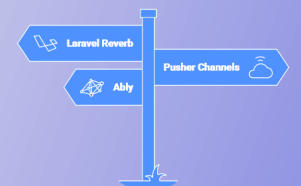




En bref

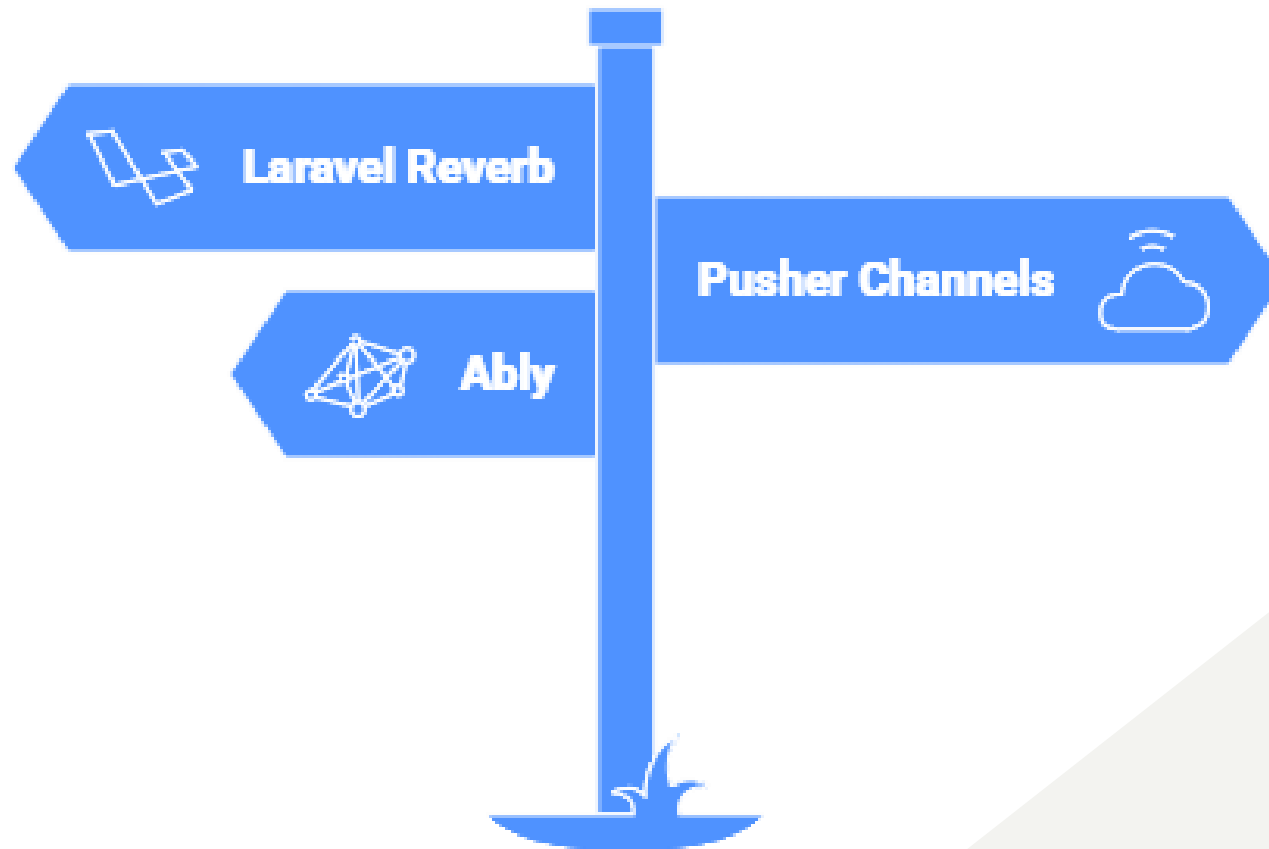
La fonctionnalité de diffusion d'événements de Laravel vous permet de diffuser vos événements Laravel côté serveur vers votre application JavaScript côté client grâce à une approche basée sur des pilotes WebSockets. Actuellement, Laravel est fourni avec les pilotes **Laravel Reverb**, **Pusher Channels** et **Ably**. Les événements peuvent être facilement traités côté client à l'aide du package JavaScript **Laravel Echo**.

Quel pilote de diffusion d'événements devrais-je utiliser ?



En bref

Quel pilote de diffusion d'événements devrais-je utiliser ?



En bref

Les événements sont diffusés via des canaux « **channels** », qui peuvent être définis comme **publics** ou **privés**. Tout visiteur de votre application peut s'abonner à un canal public sans aucune authentification ni autorisation ; en revanche, pour s'abonner à un canal privé, un utilisateur doit être authentifié et autorisé à écouter ce canal.

Introduction

Par défaut, la diffusion n'est pas activée dans les nouvelles applications Laravel. Vous pouvez l'activer à l'aide de la commande Artisan `install:broadcasting` :

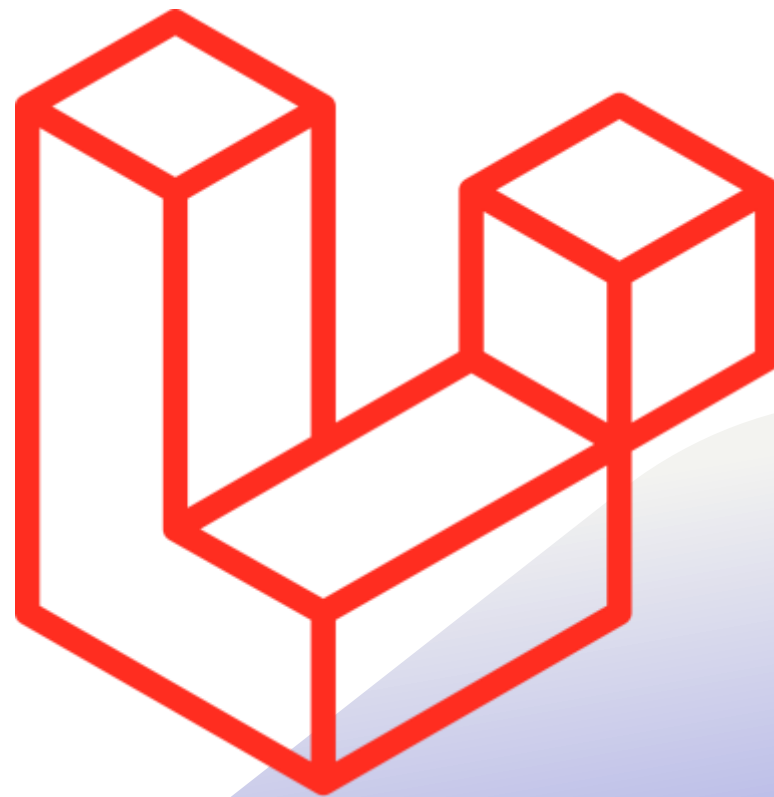
```
php artisan install:broadcasting
```


Introduction

La commande `install:broadcasting` vous demandera quel service de diffusion d'événements vous souhaitez utiliser. Elle créera également le fichier de configuration **`config/broadcasting.php`** et le fichier **`routes/channels.php`**, dans lequel vous pourrez enregistrer les routes d'autorisation de diffusion et les callbacks de votre application. Toutes les paramètres de diffusion d'événements de votre application sont enregistrés dans le fichier de configuration **`config/broadcasting.php`**. Ne vous inquiétez pas si ce fichier n'existe pas dans votre application ; il sera créé lorsque vous exécuterez la commande `Artisan install:broadcasting`.

Installation côté serveur

V9



Introduction

La diffusion d'événements est assurée par un pilote (driver) de diffusion côté serveur qui diffuse vos événements Laravel afin que **Laravel Echo** (une bibliothèque JavaScript) puisse les recevoir dans le client du navigateur.

Configuration

Toute la configuration de la diffusion d'événements de votre application est stockée dans le fichier de configuration **config/broadcasting.php**. Laravel prend en charge plusieurs pilotes de diffusion dès le départ : **Pusher Channels**, **Redis**, et **un pilote de journal** pour le développement local et le débogage. En outre, un pilote **null** est inclus qui vous permet de désactiver totalement la diffusion pendant les tests. Un exemple de configuration est inclus pour chacun de ces pilotes dans le fichier de configuration **config/broadcasting.php**.

Broadcast Service Provider

Avant de diffuser des événements, vous devez d'abord enregistrer le **App\Providers\BroadcastServiceProvider**. Dans les nouvelles applications Laravel, il vous suffit de **décommenter** ce fournisseur dans le tableau des fournisseurs de votre fichier de configuration **config/app.php**. Ce **BroadcastServiceProvider** contient le code nécessaire pour enregistrer les routes et les rappels d'autorisation de diffusion.

Canaux de Pusher

Si vous prévoyez de diffuser vos événements à l'aide de **Pusher Channels**, vous devez installer le **SDK PHP de Pusher Channels** à l'aide du gestionnaire de paquets Composer :

```
composer require pusher/pusher-php-server
```

Canaux de Pusher

Ensuite, vous devez configurer vos informations d'identification pour les canaux **Pusher** dans le fichier de configuration **config/broadcasting.php**. En général, ces configurations doivent être définies via les variables d'environnement `PUSHER_APP_KEY`, `PUSHER_APP_SECRET` et `PUSHER_APP_ID` :

```
PUSHER_APP_ID=your-pusher-app-id  
PUSHER_APP_KEY=your-pusher-key  
PUSHER_APP_SECRET=your-pusher-secret  
PUSHER_APP_CLUSTER=mt1
```

Canaux de Pusher

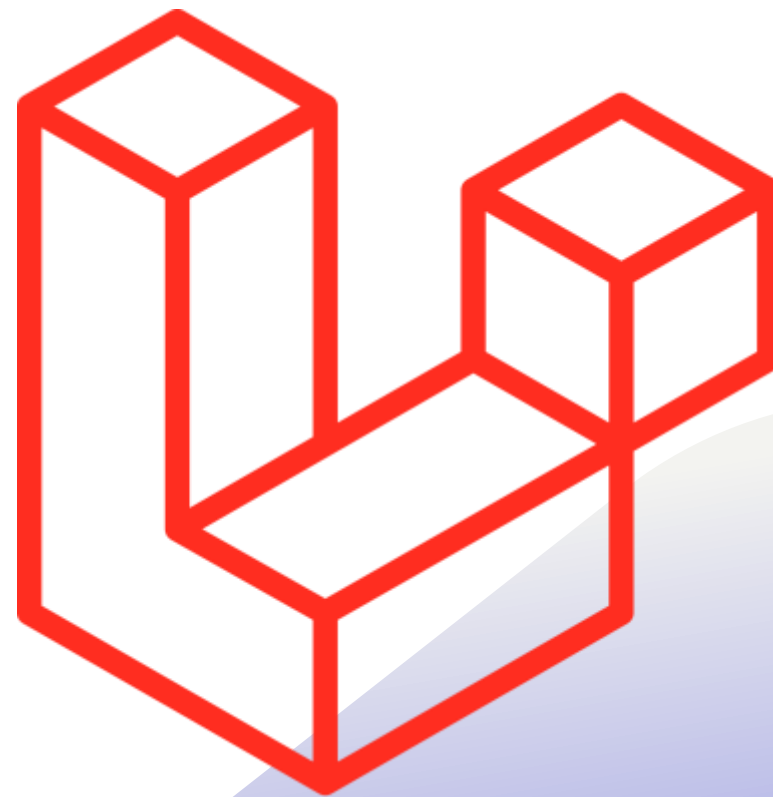
Ensuite, vous devez remplacer votre pilote de diffusion par **pusher** dans votre fichier **.env** :

```
BROADCAST_DRIVER=pusher
```

Enfin, vous êtes prêt à installer et à configurer **Laravel Echo**, qui recevra les événements de diffusion du côté client.

Installation côté serveur

V12



Introduction

La diffusion des événements est assurée par un pilote de diffusion côté serveur qui diffuse vos événements Laravel afin que Laravel Echo (une bibliothèque JavaScript) puisse les recevoir au niveau du navigateur client.

Reverb

Pour activer rapidement la prise en charge des fonctionnalités de diffusion de Laravel tout en utilisant Reverb comme diffuseur d'événements, exécutez la commande Artisan install:broadcasting avec l'option --reverb. Cette commande Artisan installera les paquets Composer et NPM requis par Reverb et mettra à jour le fichier .env de votre application avec les variables appropriées :

```
php artisan install:broadcasting --reverb
```

Reverb - Installation manuelle

Lorsque vous exécutez la commande `install:broadcasting`, vous serez invité à installer **Laravel Reverb**. Bien sûr, vous pouvez également installer **Reverb** manuellement à l'aide du gestionnaire de paquets **Composer** :

```
composer require laravel/reverb
```

Une fois le paquet installé, vous pouvez exécuter la commande d'installation de Reverb pour publier la configuration, ajouter les variables d'environnement requises par Reverb et activer la diffusion d'événements dans votre application :

```
php artisan reverb:install
```

Pusher Channels

Pour activer rapidement la prise en charge des fonctionnalités de diffusion de Laravel tout en utilisant Pusher comme diffuseur d'événements, exécutez la commande Artisan `install:broadcasting` avec l'option `--pusher`. Cette commande Artisan vous demandera vos identifiants Pusher, installera les SDK PHP et JavaScript de Pusher, et mettra à jour le fichier `.env` de votre application avec les variables appropriées :

```
php artisan install:broadcasting --pusher
```

Pusher Channels - Installation manuelle

Pour installer manuellement la prise en charge de **Pusher**, vous devez installer le SDK PHP de Pusher Channels à l'aide du gestionnaire de paquets **Composer** :

```
composer require pusher/pusher-php-server
```

Pusher Channels - Installation manuelle

Vous devez ensuite configurer vos identifiants **Pusher Channels** dans le fichier de configuration **config/broadcasting.php**. Un exemple de configuration Pusher Channels est déjà inclus dans ce fichier, ce qui vous permet de définir rapidement votre clé, votre secret et l'ID de votre application. En règle générale, vous devriez configurer vos identifiants Pusher Channels dans le fichier **.env** de votre application :

Pusher Channels - Installation manuelle

```
PUSHER_APP_ID="your-pusher-app-id"  
PUSHER_APP_KEY="your-pusher-key"  
PUSHER_APP_SECRET="your-pusher-secret"  
PUSHER_HOST=  
PUSHER_PORT=443  
PUSHER_SCHEME="https"  
PUSHER_APP_CLUSTER="mt1"
```

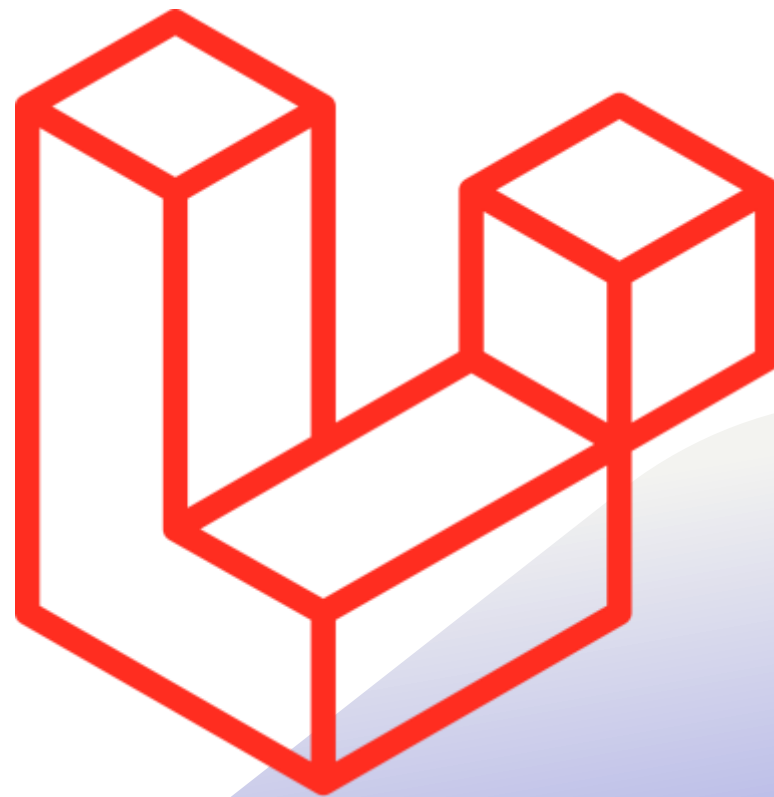

Pusher Channels - Installation manuelle

Ensuite, définissez la variable d'environnement `BROADCAST_CONNECTION` sur « pusher » dans le fichier `.env` de votre application :

```
BROADCAST_CONNECTION=pusher
```

Installation côté client

V9



Laravel Echo

Laravel Echo est une bibliothèque JavaScript qui permet de s'abonner facilement à des canaux et d'écouter les événements diffusés par votre pilote de diffusion côté serveur. Vous pouvez installer **Echo** via le gestionnaire de paquets **NPM**. Dans cet exemple, nous allons également installer le paquet `pusher-js` puisque nous allons utiliser le diffuseur **Pusher Channels** :

```
npm install --save-dev laravel-echo pusher-js
```

Laravel Echo

Une fois **Echo** installé, vous êtes prêt à créer une nouvelle instance d'Echo dans le JavaScript de votre application. Un bon endroit pour le faire est au bas du fichier **resources/js/bootstrap.js** qui est inclus dans le framework Laravel. Par défaut, un exemple de configuration d'Echo est déjà inclus dans ce fichier - il vous suffit de le décommenter :

```
import Echo from 'laravel-echo';  
import Pusher from 'pusher-js';  
  
window.Pusher = Pusher;  
  
window.Echo = new Echo({
```

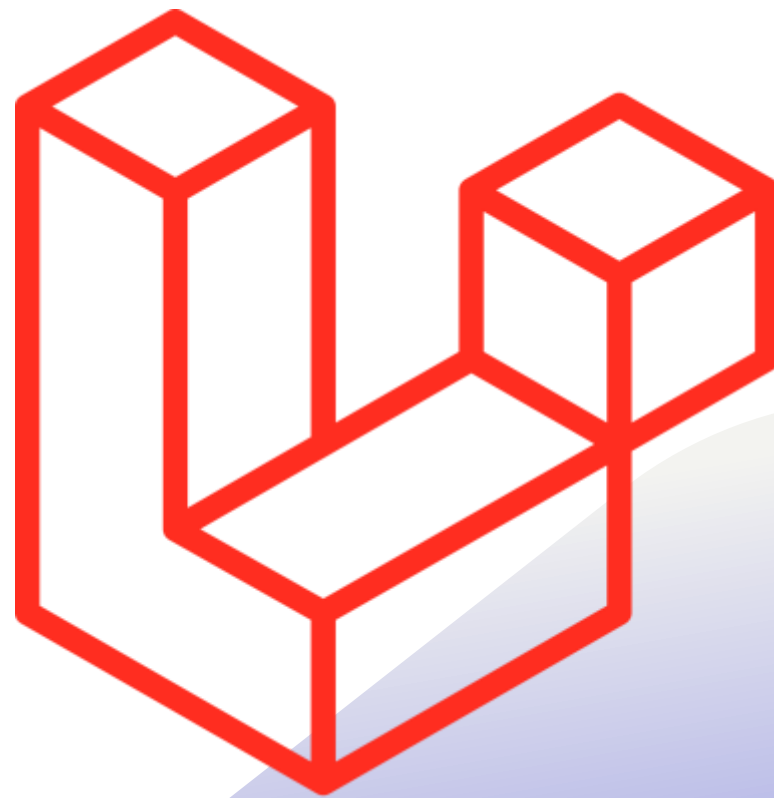
Laravel Echo

Une fois que vous avez décommenté et ajusté la configuration d'Echo en fonction de vos besoins, vous pouvez compiler les **assets** de votre application :

```
npm run dev
```

Installation côté client

V12



Reverb

Laravel Echo est une bibliothèque JavaScript qui facilite l'abonnement à des canaux et la détection des événements diffusés par votre pilote de diffusion côté serveur. Lorsque vous installez Laravel Reverb à l'aide de la commande Artisan `install:broadcasting`, la structure et la configuration de Reverb et d'Echo sont automatiquement intégrées à votre application.

Reverb - Installation manuelle

Pour configurer manuellement Laravel Echo pour l'interface utilisateur de votre application, commencez par installer le paquet **pusher-js**, car **Reverb** utilise le protocole Pusher pour les abonnements **WebSocket**, les canaux et les messages :

```
npm install --save-dev laravel-echo pusher-js
```


Reverb - Installation manuelle

Une fois Echo installé, vous êtes prêt à créer une nouvelle instance d'Echo dans le code JavaScript de votre application. L'endroit idéal pour le faire est à la fin du fichier **resources/js/bootstrap.js** fourni avec le framework Laravel :

Reverb - Installation manuelle

```
import Echo from 'laravel-echo';  
import Pusher from 'pusher-js';  
  
window.Pusher = Pusher;  
  
window.Echo = new Echo({  
  broadcaster: 'reverb',  
  key: import.meta.env.VITE_REVERB_APP_KEY,  
  wsHost: import.meta.env.VITE_REVERB_HOST,  
  wsPort: import.meta.env.VITE_REVERB_PORT ?? 80,  
  wssPort: import.meta.env.VITE_REVERB_PORT ?? 443,  
  forceTLS: (import.meta.env.VITE_REVERB_SCHEME ?? 'https') === 'https',  
  enabledTransports: ['ws', 'wss'],  
});
```

Reverb - Installation manuelle

Ensuite, vous devez compiler les ressources de votre application :

```
npm run build
```

Pusher Channels

Laravel Echo est une bibliothèque JavaScript qui facilite l'abonnement à des canaux et la détection des événements diffusés par votre pilote de diffusion côté serveur. Lorsque vous installez la prise en charge de la diffusion à l'aide de la commande Artisan `install:broadcasting --pusher`, la structure et la configuration de Pusher et d'Echo seront automatiquement intégrées à votre application.

Pusher Channels - Installation manuelle

Pour configurer manuellement Laravel Echo pour l'interface utilisateur de votre application, commencez par installer les paquets **laravel-echo** et **pusher-js**, qui utilisent le protocole Pusher pour les abonnements **WebSocket**, les canaux et les messages :

```
npm install --save-dev laravel-echo pusher-js
```

Pusher Channels - Installation manuelle

Une fois **Echo** installé, vous pouvez créer une nouvelle instance Echo dans le fichier **resources/js/bootstrap.js** de votre application :

```
import Echo from 'laravel-echo';

import Pusher from 'pusher-js';
window.Pusher = Pusher;

window.Echo = new Echo({
  broadcaster: 'pusher',
  key: import.meta.env.VITE_PUSHER_APP_KEY,
  cluster: import.meta.env.VITE_PUSHER_APP_CLUSTER,
  forceTLS: true
});
```

Pusher Channels - Installation manuelle

Vous devez ensuite définir les valeurs appropriées pour les variables d'environnement Pusher dans le fichier **.env** de votre application. Si ces variables n'existent pas encore dans votre fichier **.env**, vous devez les ajouter :

```
PUSHER_APP_ID="your-pusher-app-id"
PUSHER_APP_KEY="your-pusher-key"
PUSHER_APP_SECRET="your-pusher-secret"
PUSHER_HOST=
PUSHER_PORT=443
PUSHER_SCHEME="https"
PUSHER_APP_CLUSTER="mt1"

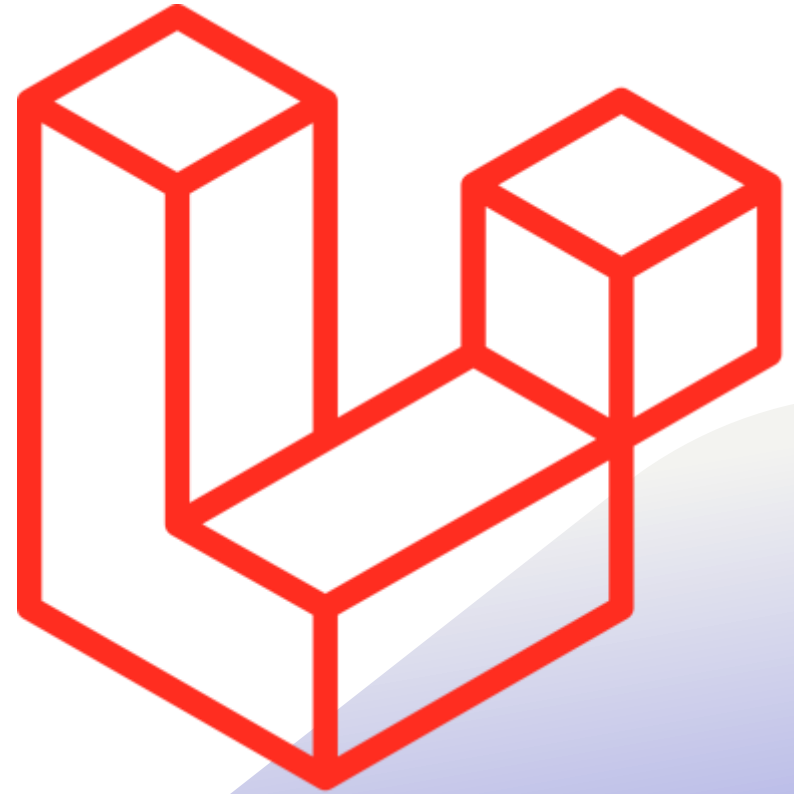
VITE_APP_NAME="${APP_NAME}"
VITE_PUSHER_APP_KEY="${PUSHER_APP_KEY}"
VITE_PUSHER_HOST="${PUSHER_HOST}"
VITE_PUSHER_PORT="${PUSHER_PORT}"
VITE_PUSHER_SCHEME="${PUSHER_SCHEME}"
VITE_PUSHER_APP_CLUSTER="${PUSHER_APP_CLUSTER}"
```

Pusher Channels - Installation manuelle

```
PUSHER_APP_ID="your-pusher-app-id"  
PUSHER_APP_KEY="your-pusher-key"  
PUSHER_APP_SECRET="your-pusher-secret"  
PUSHER_HOST=  
PUSHER_PORT=443  
PUSHER_SCHEME="https"  
PUSHER_APP_CLUSTER="mt1"
```

```
VITE_APP_NAME="${APP_NAME}"  
VITE_PUSHER_APP_KEY="${PUSHER_APP_KEY}"  
VITE_PUSHER_HOST="${PUSHER_HOST}"  
VITE_PUSHER_PORT="${PUSHER_PORT}"  
VITE_PUSHER_SCHEME="${PUSHER_SCHEME}"  
VITE_PUSHER_APP_CLUSTER="${PUSHER_APP_CLUSTER}"
```


Aperçu du concept via un exemple



Utilisation d'un exemple d'application

Prenons une boutique de commerce électronique comme exemple.

Dans notre application, supposons que nous avons une page qui permet aux utilisateurs de voir l'état d'expédition de leurs commandes. Supposons également qu'un événement **OrderShipmentStatusUpdated** est déclenché lorsqu'une mise à jour de l'état d'expédition est traitée par l'application :

```
use App\Events\OrderShipmentStatusUpdated;  
OrderShipmentStatusUpdated::dispatch($order);
```

L'interface ShouldBroadcast

Lorsqu'un utilisateur consulte l'une de ses commandes, nous ne voulons pas qu'il ait à rafraîchir la page pour voir les mises à jour d'état. Au lieu de cela, nous voulons diffuser les mises à jour à l'application dès qu'elles sont créées. Nous devons donc marquer l'événement **OrderShipmentStatusUpdated** avec l'interface **ShouldBroadcast**. Cela permettra à Laravel de diffuser l'événement lorsqu'il est déclenché :

```
class OrderShipmentStatusUpdated implements ShouldBroadcast
{
    public $order;
}
```

L'interface ShouldBroadcast

L'interface **ShouldBroadcast** exige que notre événement définisse une méthode **broadcastOn**. Cette méthode est chargée de renvoyer les canaux sur lesquels l'événement doit être diffusé. Nous voulons seulement que le créateur de la commande puisse voir les mises à jour de statut, donc nous diffuserons l'événement sur un canal privé qui est lié à la commande :

```
public function broadcastOn()  
{  
    return new PrivateChannel('orders.'.$this->order->id);  
}
```

Autorisation des canaux

N'oubliez pas que les utilisateurs doivent être autorisés à écouter les canaux privés. Nous pouvons définir nos règles d'autorisation des canaux dans le fichier **routes/channels.php** de notre application. Dans cet exemple, nous devons vérifier que tout utilisateur qui tente d'écouter le canal privé **orders.1** est bien le créateur de la commande :

```
Broadcast::channel('orders.{orderId}', function ($user, $orderId) {  
    return $user->id === Order::findOrCreate($orderId)->user_id;  
});
```

Autorisation des canaux

La méthode **channel** accepte deux arguments : le nom du canal et un callback qui renvoie **true** ou **false** indiquant si l'utilisateur est autorisé à écouter sur le canal.

Tous les rappels d'autorisation reçoivent l'utilisateur actuellement authentifié comme premier argument et tous les paramètres supplémentaires comme arguments suivants. Dans cet exemple, nous utilisons le caractère générique **{orderId}** pour indiquer que la partie "ID" du nom du canal est un caractère générique.

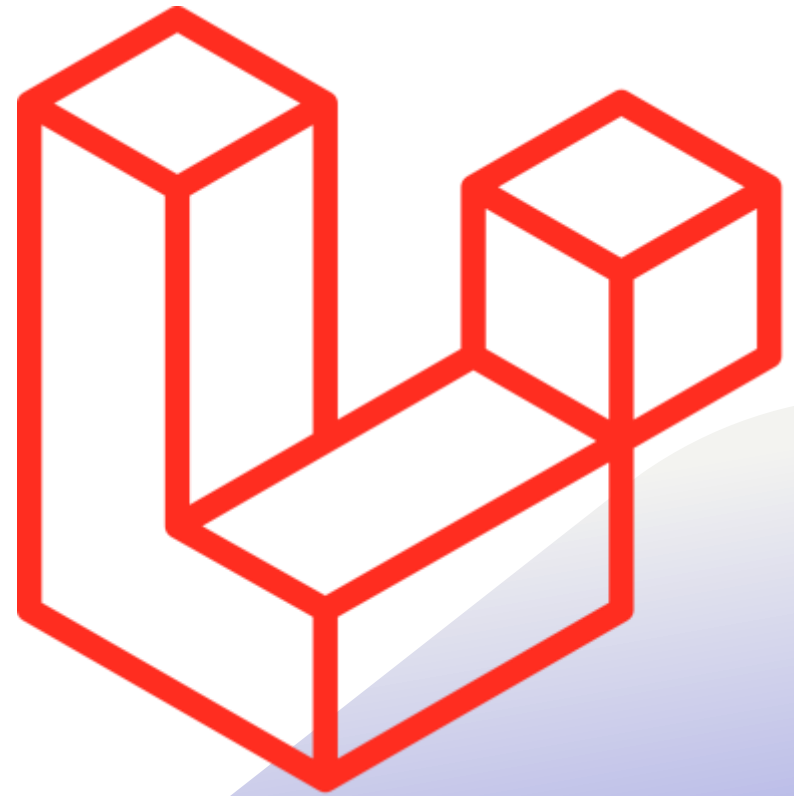
Écouter les diffusions d'événements

Ensuite, il ne reste plus qu'à écouter l'événement dans notre application JavaScript. Nous pouvons le faire en utilisant **Laravel Echo**. Tout d'abord, nous allons utiliser la méthode **private** pour nous abonner au canal privé. Ensuite, nous pouvons utiliser la méthode **listen** pour écouter l'événement **OrderShipmentStatusUpdated**.

Par défaut, toutes les propriétés publiques de l'événement seront incluses dans l'événement de diffusion :

```
Echo.private(`orders.${orderId}`)  
  .listen('OrderShipmentStatusUpdated', (e) => {  
    console.log(e.order);  
  });
```

Définition des événements de diffusion



Définition des événements de diffusion

Pour informer Laravel qu'un événement donné doit être diffusé, vous devez implémenter l'interface

Illuminate\Contracts\Broadcasting\ShouldBroadcast sur la classe d'événement. Cette interface est déjà importée dans toutes les classes d'événements générées par le framework, vous pouvez donc facilement l'ajouter à n'importe lequel de vos événements.

Définition des événements de diffusion

L'interface **ShouldBroadcast** exige que vous implémentiez une seule méthode : **broadcastOn**.

La méthode **broadcastOn** doit renvoyer un canal ou un tableau de canaux sur lesquels l'événement doit être diffusé. Les canaux doivent être des instances de **Channel**, **PrivateChannel** ou **PresenceChannel**.

Les instances de **Channel** représentent des canaux publics auxquels tout utilisateur peut s'abonner, tandis que **PrivateChannels** et **PresenceChannels** représentent des canaux privés qui nécessitent une autorisation.

Définition des événements de diffusion

```
class ServerCreated implements ShouldBroadcast
{
    use SerializesModels;
    public $user;
    public function __construct(User $user)
    {
        $this->user = $user;
    }
    public function broadcastOn()
    {
        return new PrivateChannel('user.'.$this->user->id);
    }
}
```

Nom de diffusion

Par défaut, Laravel diffuse l'événement en utilisant le nom de la classe de l'événement. Cependant, vous pouvez personnaliser le nom de diffusion en définissant une méthode **broadcastAs** sur l'événement :

```
public function broadcastAs()  
{  
    return 'server.created';  
}
```

Nom de diffusion

Si vous personnalisez le nom de la diffusion à l'aide de la méthode **broadcastAs**, vous devez vous assurer d'enregistrer votre écouteur avec un caractère . en tête. Cela permettra à Echo de ne pas ajouter l'espace de nom de l'application à l'événement :

```
.listen('.server.created', function (e) {  
    ....  
});
```

Données de diffusion

Lorsqu'un événement est diffusé, toutes ses propriétés publiques sont automatiquement sérialisées et diffusées en tant que charge utile de l'événement, ce qui vous permet d'accéder à toutes ses données publiques depuis votre application JavaScript. Ainsi, par exemple, si votre événement a une seule propriété publique **\$user** qui contient un modèle Eloquent, la charge utile de diffusion de l'événement serait la suivante :

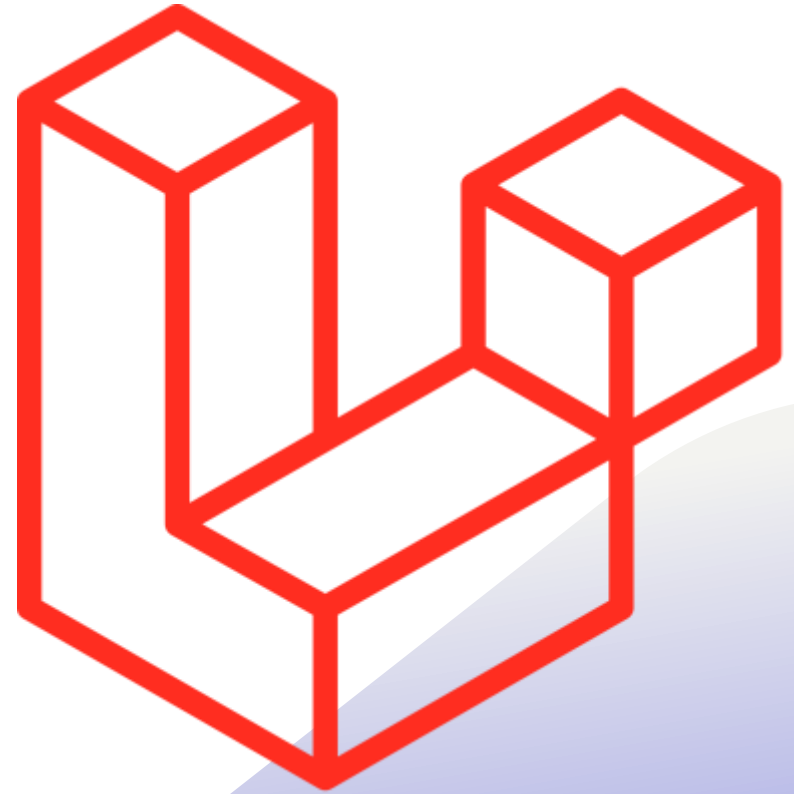
```
{  
  "user": {  
    "id": 1,  
    "name": "Patrick Stewart"  
    ...  
  }  
}
```

Données de diffusion

Cependant, si vous souhaitez avoir un contrôle plus fin de votre charge utile de diffusion, vous pouvez ajouter une méthode **broadcastWith** à votre événement. Cette méthode doit renvoyer le tableau de données que vous souhaitez diffuser comme charge utile de l'événement :

```
public function broadcastWith()  
{  
    return ['id' => $this->user->id];  
}
```

Événements de diffusion



Évènements de diffusion

Une fois que vous avez défini un événement et que vous l'avez marqué avec l'interface **ShouldBroadcast**, il vous suffit de déclencher l'événement en utilisant la méthode de distribution de l'événement. Le répartiteur d'événements remarquera que l'événement est marqué par l'interface **ShouldBroadcast** et mettra l'événement en file d'attente pour diffusion :

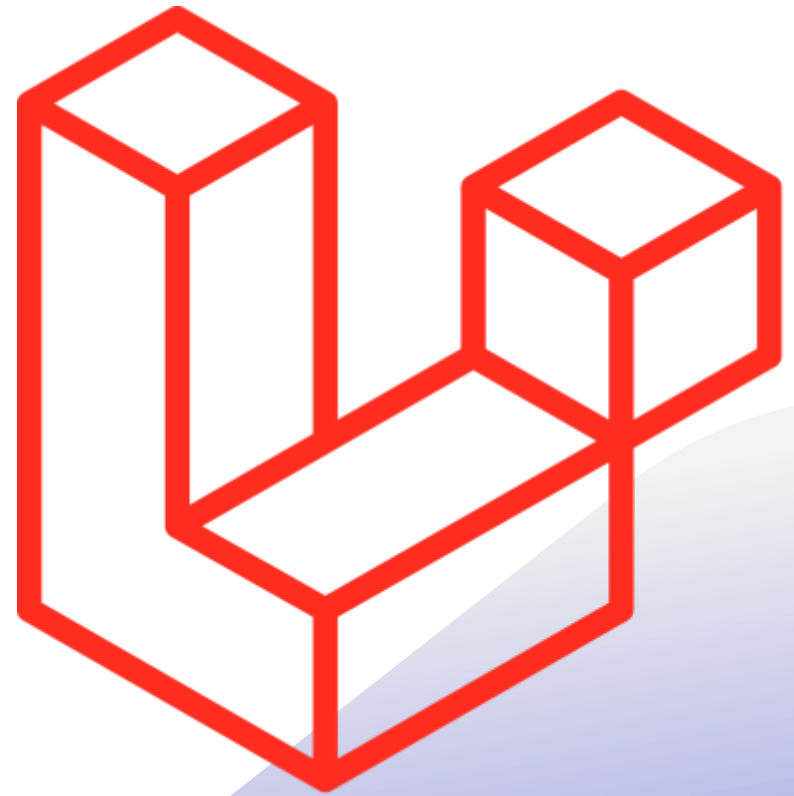
```
OrderShipmentStatusUpdated::dispatch($order);
```

Personnalisation de la connexion

Si votre application interagit avec plusieurs connexions de diffusion et que vous souhaitez diffuser un événement en utilisant un diffuseur autre que celui par défaut, vous pouvez spécifier la connexion vers laquelle pousser un événement à l'aide de la méthode **via** :

```
broadcast(new OrderShipmentStatusUpdated($update))->via('pusher');
```

Réception des diffusions



Écoute des événements

Une fois que vous avez installé et instancié **Laravel Echo**, vous êtes prêt à commencer à écouter les événements qui sont diffusés depuis votre application Laravel. Tout d'abord, utilisez la méthode **channel** pour récupérer une instance d'un canal, puis appelez la méthode **listen** pour écouter un événement spécifié :

```
Echo.channel(`orders.${this.order.id}`)  
  .listen('OrderShipmentStatusUpdated', (e) => {  
    console.log(e.order.name);  
  });
```

Écoute des événements

Si vous souhaitez écouter des événements sur un canal privé, utilisez plutôt la méthode **private**. Vous pouvez continuer à enchaîner les appels à la méthode **listen** pour écouter plusieurs événements sur un seul canal :

```
Echo.private(`orders.${this.order.id}`)  
  .listen(/* ... */)   
  .listen(/* ... */)   
  .listen(/* ... */);
```

Arrêtez d'écouter les événements

Si vous souhaitez arrêter d'écouter un événement donné sans quitter le canal, vous pouvez utiliser la méthode **stopListening** :

```
Echo.private(`orders.${this.order.id}`)  
  .stopListening('OrderShipmentStatusUpdated')
```

Quitter un canal

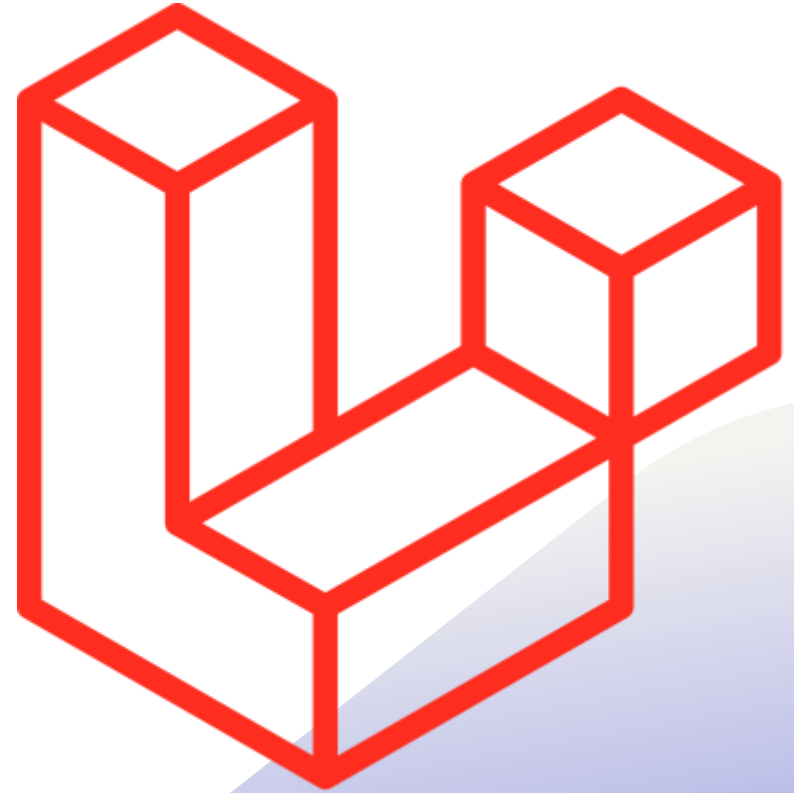
Pour quitter un canal, vous pouvez appeler la méthode **leaveChannel** sur votre instance **Echo** :

```
Echo.leaveChannel(`orders.${this.order.id}`);
```

Si vous souhaitez quitter un canal ainsi que les canaux privés et de présence qui lui sont associés, vous pouvez appeler la méthode **leave** :

```
Echo.leave(`orders.${this.order.id}`);
```

Diffusion de modèles



Diffusion de modèles

Il est courant de diffuser des événements lorsque les modèles Eloquent de votre application sont créés, mis à jour ou supprimés. Bien sûr, ceci peut être facilement accompli en définissant manuellement des événements personnalisés pour les changements d'état des modèles Eloquent et en marquant ces événements avec l'interface **ShouldBroadcast**.

Diffusion de modèles

Pour commencer, votre modèle Eloquent doit utiliser le trait **Illuminate\Database\Eloquent\BroadcastsEvents**. En outre, le modèle doit définir une méthode **broadcastOn**, qui renvoie un tableau de canaux sur lesquels les événements du modèle doivent être diffusés :

```
class Post extends Model
{
    use BroadcastsEvents, HasFactory;
    //...
    public function broadcastOn($event)
    {
        return [$this, $this->user];
    }
}
```

Diffusion de modèles

Une fois que votre modèle inclut ce trait et définit ses canaux de diffusion, il commencera à diffuser automatiquement des événements lorsqu'une instance de modèle est créée, mise à jour, supprimée, mise à la corbeille ou restaurée.

Diffusion de modèles

La méthode **broadcastOn** reçoit un argument de type chaîne **\$event**. Cet argument contient le type d'événement qui s'est produit sur le modèle et aura pour valeur **created**, **updated**, **deleted**, **trashed**, ou **restored**.

```
public function broadcastOn($event)
{
    return match ($event) {
        'deleted' => [],
        default => [$this, $this->user],
    };
}
```